

EDS Practical 1

Name:- Avadhoot R. Huddar

Batch:-M.E.21

PRN:- 202501090087

1.1.1. Calculate Momentum

09:12

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p=m*v$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

A single floating-point number representing the mass of the object in kilograms.

A single floating-point number representing the velocity of the object in meters per second.

Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Input:-

```
m=float(input())
```

```
v=float(input())
```

```
p=m*v
```

```
print(f"{p:.2f}kgm/s")
```

CODETANTRA Home Learn Anywhere

202501090087@mitaoe.ac.in Support Logout

Essentials of Data Science Laboratory - 2304102L - 2026

Search course **ctrl + k**

1. Practical 1

1.1. Practice Lab Assignment

1.1.1. Calculate Momentum

1.1.2. Conditional Calculation Based on the...

1.1.3. Days between Two Dates

1.1.4. Reverse a Number

1.1.5. Multiplication Table

1.2. Lab Assignment

2. Practical 2

3. Practical 3

4. Practical 4

5. Practical 5

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

- m is the mass of the object (in kilograms).
- v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

calculate...

```

1 m=float(input())
2 v=float(input())
3 p=m*v
4 print(f"{p:.2f}kgm/s")

```

Average time: 0.015 s, 14.75 ms | Maximum time: 0.027 s, 27.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1

Expected output	Actual output
5.0	5.0
10.0	10.0
50.00kgm/s	50.00kgm/s

Test case 2

Expected output	Actual output
2.3	2.3
4.8	4.8
11.04kgm/s	11.04kgm/s

Terminal Test cases

< Prev Reset Submit Next >

1.1.2. Conditional Calculation Based on the Number of Digits

09:16

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraints:

$$1 \leq \leq 999$$

Input Format:

The input consists of a single integer .

Output Format:

If is a single-digit number, print its square.

If is a two-digit number, print its square root (rounded to two decimal places).

If is a three-digit number, print its cube root (rounded to two decimal places).

```
Else print "Invalid".
```

Input:-

```
import math
```

```
n=int(input())
```

```
if 1<=n<=9:
```

```
    print(n**2)
```

```
elif 10<=n<=99:
```

```
    print(f"{math.sqrt(n):.2f}")
```

```
elif 100<=n<=999:
```

```
    print(f"{math.cbrt(n):.2f}")
```

```
else:
```

```
    print("Invalid")
```

The screenshot displays the CODETANTRA web interface. On the left, a sidebar lists course topics under 'Essentials of Data Science Laboratory - 2304102L - 2026'. The main content area shows a lab assignment titled '1.1.2. Conditional Calculation Based on the Number of Digits'. It includes constraints, input/output formats, and sample test cases. On the right, a code editor shows the Python code for the assignment. Below the code, a table displays the execution results for four test cases, showing expected and actual outputs, and a summary of test case passes.

Test Case	Expected Output	Actual Output
Test case 1	81	81
Test case 2	166	166
Test case 3	24	24
Test case 4	3456	3456

1.1.3. Days between Two Dates

03:58

Write a Python program to calculate number of days between two dates.

Input Format:

The first line contains the first date in the format .

The second line contains the second date in the format .

Output Format:

An integer representing the number of days between the given dates.

Note:

The first date should always be considered earlier than the second date.

Input:-

```
from datetime import date
```

```
from datetime import datetime
```

```
date1=input()
```

```
date2=input()
```

```
d1=datetime.strptime(date1, "%Y-%m-%d")
```

```
d2=datetime.strptime(date2, "%Y-%m-%d")
```

```
difference=d2-d1
```

```
print(difference.days)
```

CODETANTRA Home Learn Anywhere 202501090087@mitaoe.ac.in Support Logout

Essentials of Data Science Laboratory - 2304102L - 2026

Search course **ctrl + k**

- 1. Practical 1
 - 1.1. Practice Lab Assignment
 - 1.1.1. Calculate Momentum
 - 1.1.2. Conditional Calculation Based on the...
 - 1.1.3. Days between Two Dates**
 - 1.1.4. Reverse a Number
 - 1.1.5. Multiplication Table
 - 1.2. Lab Assignment
- 2. Practical 2
- 3. Practical 3
- 4. Practical 4
- 5. Practical 5

1.1.3. Days between Two Dates 03.5s

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format `YYYY-MM-DD`.
- The second line contains the second date in the format `YYYY-MM-DD`.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

Test case 1

2014-07-02	2014-11-02
123	123

Test case 2

2014-07-02	2014-07-11
9	9

noOfDays... Submit

```
1 from datetime import date
2 from datetime import datetime
3 date1=input()
4 date2=input()
5 d1=datetime.strptime(date1, "%Y-%m-%d")
6 d2=datetime.strptime(date2, "%Y-%m-%d")
7 difference=d2-d1
8 print(difference.days)
```

Average time: 0.032 s, 32.25 ms | Maximum time: 0.078 s, 78.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1: Expected output: 2014-07-02, 2014-11-02 | Actual output: 2014-07-02, 2014-11-02 | 123 | 123

Test case 2: Expected output: 2014-07-02, 2014-07-11 | Actual output: 2014-07-02, 2014-07-11 | 9 | 9

Terminal Test cases

< Prev Reset Submit Next >

1.1.4. Reverse a Number

07:17

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

The input is a single integer.

Output Format:

The output prints a single integer, which is the reversed number.

Input:-

```
n=int(input())
```

```
reverse=int(str(n)[::-1])
```

```
print(reverse)
```

CODETANTRA Home Learn Anywhere

202501090087@mitaoe.ac.in Support Logout

Essentials of Data Science Laboratory - 2304102L - 2026

Search course **ctrl + k**

1. Practical 1

1.1. Practice Lab Assignment

1.1.1. Calculate Momentum

1.1.2. Conditional Calculation Based on the...

1.1.3. Days between Two Dates

1.1.4. Reverse a Number

1.1.5. Multiplication Table

1.2. Lab Assignment

2. Practical 2

3. Practical 3

4. Practical 4

5. Practical 5

1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases

Test case 1

Expected output: 5367

Actual output: 5367

7635

Test case 2

Expected output: 8132

Actual output: 8132

231

reverseN...

```
1 n=int(input())
2 reverse=int(str(n)[::-1])
3 print(reverse)
```

Average time: 0.008 s, Maximum time: 0.009 s

7.80 ms, 9.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1: Expected output: 5367, Actual output: 5367

Test case 2: Expected output: 8132, Actual output: 8132

Terminal Test cases

< Prev Reset Submit Next >

1.1.5. Multiplication Table

07:43

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

The character "x" is used in the output to represent multiplication.

Refer to the visible test-cases for more clarity.

Input:-

```
n=int(input())

for i in range(1,11):

    print(f"{n} x {i} = {n*i}")
```

The screenshot shows the CODETANTRA IDE interface. On the left is a sidebar with a search bar and a list of courses under 'Essentials of Data Science Laboratory - 2304102L - 2026'. The main area is divided into three parts: a file explorer showing '1.1.5. Multiplication Table', a code editor with the Python code, and a test cases panel. The code in the editor is:

```
n=int(input())
for i in range(1,11):
    print(f'{n} x {i} = {n*i}')
```

The test cases panel shows two test cases, both of which passed. The first test case is for n=8 and the second is for n=5. The expected output and actual output for each test case are shown.

1.2.1. Pass or Fail

20:39

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

If the aggregate percentage is greater than 75, the grade is Distinction.

If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.

If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.

If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

The first input will be an integer , the number of courses.

The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

Print the aggregate percentage (formatted to two decimal places).

Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

"Fail".

Note:

Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Input:-

```
def cal(marks,total_courses):
```

```
    if any(mark<40 for mark in marks):
```

```
        return "FAIL"
```

```
    total_marks = sum(marks)
```

```
    aggregate_percentage=(total_marks/(total_courses*100))*100
```

```
    if aggregate_percentage>75:
```

```
        grade="Distinction"
```

```
    elif aggregate_percentage>=60:
```

```
        grade="First Division"
```

```
    elif aggregate_percentage>=50:
```

```
        grade="Second Division"
```

```
    elif aggregate_percentage>=40:
```

```
grade="Third Division"
```

```
return(aggregate_percentage,grade)
```

```
num_courses=int(input())
```

```
marks=list(map(int,input().split()))
```

```
result=cal(marks,num_courses)
```

```
if result == "FAIL":
```

```
    print("Fail")
```

```
else:
```

```
    aggregate_percentage, grade = result
```

```
    print(f"Aggregate Percentage: {aggregate_percentage:.2f}")
```

```
    print(f"Grade: {grade}")
```

The screenshot displays the CODETANTRA web application interface. On the left, a sidebar menu lists various lab assignments under the heading 'Essentials of Data Science Laboratory - 2304102L - 2026'. The main content area shows a list of test cases for a specific assignment. Each test case includes an input field, a 'Run' button, and a 'Debug' button. The results section on the right shows the expected output, actual output, and the resulting grade for each test case. The interface is clean and modern, with a dark theme and clear typography.

1.2.2. Fibonacci Series

02:23

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

A single integer representing the number of terms to generate.

Output Format:

A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Input:-

```
def fibonacci(n):  
    if n==1:  
        return 0  
    elif n==2:  
        return 1  
    else:  
        return fibonacci(n-1)+fibonacci(n-2)  
  
n = int(input())  
for i in range(1, n + 1):  
    print(fibonacci(i), end=" ")
```

Essentials of Data Science Laboratory - 2304102L - 2026

1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

Test case 1

Expected output: 0 1 1 2 3

Actual output: 0 1 1 2 3

Test case 2

Expected output: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Actual output: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

```
def fibonacci(n):  
    if n==1:  
        return 0  
    elif n==2:  
        return 1  
    else:  
        return fibonacci(n-1)+fibonacci(n-2)  
  
n = int(input())  
for i in range(1, n + 1):  
    print(fibonacci(i), end=" ")
```

Test results: 2 out of 2 shown test case(s) passed, 2 out of 2 hidden test case(s) passed.

1.2.3. Pattern - 1

11:01

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format

The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Input:-

```
n=int(input())
```

```
for i in range(1,n+1):
```

```
    for j in range(i):
```

```
print("*",end=" ")
```

```
print()
```

Essentials of Data Science Laboratory - 2304102L - 2026

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

Test case 1

Expected output

```
1
1 1
1 1 1
1 1 1 1
1 1 1 1 1
```

Actual output

```
1
1 1
1 1 1
1 1 1 1
1 1 1 1 1
```

Test case 2

Expected output

```
1
1 1
1 1 1
1 1 1 1
1 1 1 1 1
```

Actual output

```
1
1 1
1 1 1
1 1 1 1
1 1 1 1 1
```

Average time: 0.009 s, Maximum time: 0.011 s, 2 out of 2 shown test case(s) passed, 4 out of 4 hidden test case(s) passed

1.2.4. Pattern - 2

06:26

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format:

The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

Refer to the displayed test cases for the sample pattern.

Input:-

```
n=int(input())
```

```

for i in range(1,n+1):

    for j in range(1,i+1):

        print(j,end=" ")

    print()

```

The screenshot displays the CODETANTRA web interface. On the left, a sidebar lists various practice problems under 'Essentials of Data Science Laboratory - 2304102L - 2026'. The main content area shows the details for '1.2.4. Pattern - 2', including input/output formats, sample test cases, and a note. The right panel shows the code editor with a Python solution and the test case results, indicating that 2 out of 2 shown test cases passed.

1.2.4. Pattern - 2

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

Test case 1

5

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

Test case 2

8

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8

```

numberP...

```

1 n=int(input())
2 for i in range(1,n+1):
3     for j in range(1,i+1):
4         print(j,end=" ")
5     print()

```

Test case 1 (13 ms)

Expected output: 5

Actual output: 5

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

Test case 2 (13 ms)

Expected output: 8

Actual output: 8

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8

```

Terminal **Test cases**

< Prev Reset Submit Next >